

Wave 5.2R Stage 2 Evaluation And Optimization Instrumentation Report

Feedforward Transmission Error Baseline

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Executive Decision

Wave 5.2R Stage 2 passes its implementation and deterministic smoke gate.

The repository can now observe the optimization interaction among a protected data-fit loss and any number of curve, harmonic, derivative, or weak-physics losses before those terms are used to justify a PINN result.

The implemented layer provides

- named loss components with explicit raw units and normalization scales;
- per-step raw values, normalized values, and exponential moving averages;
- a gradient norm for every loss on declared shared parameters;
- pairwise gradient cosine similarity;
- an optimizer update-to-parameter ratio;
- fixed, gradient-statistics, ReLoBRaLo-style, and conflict-aware adapters;
- staged loss activation and parameter freeze-unfreeze schedules;
- deterministic seed and dataloader helpers with exact batch fingerprints;
- a main-loss-preserving auxiliary-gradient projection.

All twelve exit-gate checks pass. The validation deliberately created an auxiliary weak-physics gradient that opposed the protected data gradient:

- cosine before projection: `-0.9548379` ;
- cosine after projection: approximately `0.0000002` ;
- projection applied: `true` ;
- optimizer update-to-parameter ratio: `0.4569128` .

These values prove that the diagnostics see a severe conflict and that the conflict-aware adapter removes only the opposing auxiliary component. They are synthetic instrumentation checks, not model-accuracy results. No project-data training was executed. Existing campaign behavior remains unchanged until a later, separately prepared campaign explicitly adopts the new utility.

Stage Scope

Included Contract

The instrumentation is designed for the frozen Wave 5.2R lane:

- dataset: `polished_dataset` ;
- input mode: `setpoints` ;
- surface: `Fw` ;
- split signature:
`c1aa8718fb9bf88cc2021c121dc4f3b4010fc1d2e45ac90af5f4376aa64f8e16` ;
- accepted time-windowed reference:
`polished_setpoints_periodic_gru_sequence_Fw` ;
- accepted non-windowed reference:
`polished_setpoints_periodic_mlp_harmonic_Fw` ;
- accepted analytical reference: `PF_A_LOCAL_QUADRATIC` .

The Stage 2 harness does not read or modify the frozen split. It validates generic optimization behavior on synthetic tensors so Stage 3 can stress the analytical anchor without first conflating formulation quality and optimizer pathology.

Excluded Work

Stage 2 does not

- train a candidate on the polished dataset;
- change an accepted reference;
- claim that any physical equation is informative;
- promote an adaptive weighting method;
- modify a historical campaign;
- enable gradient surgery in existing Lightning training by default;
- make a cross-platform bitwise reproducibility claim;
- replace the future matched model campaigns.

Why This Instrumentation Is Necessary

A composite PINN objective often has the form:

Total loss: data loss + shape loss + harmonic loss + derivative loss + physics residual + boundary loss.

The scalar total alone hides four distinct failure modes.

Unit Imbalance

Two losses can have different natural scales even when their numerical coefficients are equal. A fixed coefficient of one therefore does not imply equal training pressure.

Gradient Magnitude Imbalance

Two normalized loss values can have similar magnitudes while producing very different parameter-gradient norms. The larger gradient can dominate the update.

Gradient Conflict

An auxiliary prior can decrease only by increasing the protected prediction loss. This appears as negative cosine similarity between the two gradients.

Schedule Instability

A valid auxiliary objective can still destabilize early optimization if it is fully active before the data representation or analytical anchor has learned a useful state.

Stage 2 makes each failure mode directly observable.

Implemented Architecture

Component Contract

Each `LossComponentConfiguration` declares

Field	Meaning
<code>name</code>	Stable machine-facing loss identifier
<code>unit_label</code>	Raw physical or normalized unit description
<code>normalization_scale</code>	Positive divisor for comparison scaling
<code>fixed_weight</code>	Baseline coefficient used by matched controls
<code>role</code>	Protected main loss or auxiliary loss
<code>activation_schedule</code>	Start, warm-up, and optional end steps

Exactly one component must be declared as the protected main loss. This prevents an adapter from silently changing which objective has priority.

Unit Normalization

For raw component loss `Li` and declared positive scale `si`, the comparison loss is:

Normalized loss: `Liraw / si`.

The scale is explicit configuration. It must be fitted or selected using the training partition only in future project-data campaigns.

The instrumentation does not infer units from target data and does not reinterpret a physics residual as TE error.

Loss Exponential Moving Average

For normalized loss `Li(t)`, retention coefficient `beta`, and previous moving average `Ei(t-1)`:

Loss EMA: `beta * Ei(t-1) + (1 - beta) * Li(t)`.

The first observation initializes the moving average directly. This makes the record interpretable from step zero and avoids an artificial initial bias toward zero.

Functional Per-Loss Gradients

For shared parameters `theta`

Component gradient: gradient of `L_i_normalized` with respect to `theta`.

The implementation uses functional `torch.autograd.grad` with retained graph and explicit handling of unused parameters. It does not call `backward` and does not populate parameter `.grad` buffers.

This separation matters because diagnostics must not alter the optimizer state they are supposed to measure.

Gradient Norm

For flattened component gradient `g_i`

Gradient norm: L2 norm of `g_i`.

The norm measures training pressure, not physical validity. A large norm can indicate useful signal, poor normalization, numerical instability, or an incorrect residual.

Pairwise Gradient Cosine

For two component gradients

Gradient cosine: $\text{dot}(g_i, g_j) / (\text{norm}(g_i) * \text{norm}(g_j))$.

Interpretation

Cosine	Meaning
close to <code>+1</code>	strongly aligned update directions
close to <code>0</code>	locally independent or orthogonal directions
below <code>0</code>	objectives conflict on the shared parameters

Zero-norm pairs are reported as zero rather than producing undefined values. The corresponding gradient norms remain visible, so a zero gradient cannot be mistaken for genuine orthogonality.

Update-To-Parameter Ratio

For parameter vector `theta` before and after an optimizer step:

Update ratio: $\text{norm}(\text{theta_after} - \text{theta_before}) / \text{norm}(\text{theta_before})$.

The dimensionless ratio reveals an optimizer step that is negligible, disproportionately large, or inconsistent across adapters.

Implemented Adapter Modes

Fixed

The fixed adapter returns the declared component coefficients multiplied by their schedule values.

It supports both required fixed controls

- equal raw weights use identity normalization scales;
- manually normalized fixed weights use declared component scales.

The same adapter therefore isolates normalization from adaptive weighting.

Gradient Statistics

The protected main gradient defines the reference norm. For auxiliary component i :

Auxiliary weight: $\text{fixed_weight_i} * \text{clamp}(\text{main_gradient_norm} / \text{gradient_norm_i})$.

The ratio is clamped to explicit positive limits. The main coefficient is unchanged.

This adapter can balance gradient magnitude, but it cannot determine whether a physical equation is correct.

ReLoBRaLo-Style Relative Balancing

The implementation tracks

- initial normalized loss;
- previous normalized loss;
- previous component weight;
- deterministic seeded lookback selection.

Relative progress is computed against either the initial or previous state, converted to a temperature-scaled balanced weight, and blended with the previous weight. The name deliberately includes `style` . This is a repository-owned, deterministic implementation of the relative-progress principle, not a claim of byte-for-byte reproduction of every reference implementation detail.

Conflict Aware

The conflict-aware mode preserves the main gradient and examines each auxiliary gradient independently.

If

```
dot(g_auxiliary, g_main) < 0
```

the auxiliary gradient becomes

Projected auxiliary gradient: $\text{g_auxiliary} - \text{dot}(\text{g_auxiliary}, \text{g_main}) / \text{squared_norm}(\text{g_main}) * \text{g_main}$.

The combined update is

Combined gradient: $\text{weight_main} * \text{g_main} + \text{sum}(\text{weight_i} * \text{g_i_projected})$.

The protected main gradient is never projected. The caller must explicitly assign the returned flat vector and execute the optimizer step.

This design avoids hidden gradient manipulation inside unchanged Lightning automatic optimization.

Staged Activation And Freezing

Loss Activation

`LossActivationSchedule` supports

- zero weight before `start_step` ;
- linear warm-up from `start_step` to `full_weight_step` ;
- full weight afterward;
- optional deactivation at `end_step` .

The Stage 2 physics component uses:

Step	Multiplier
0	0.0
2	0.0
4	0.5
6	1.0
7	1.0

Parameter Freeze-Unfreeze

`ParameterFreezeSchedule` matches explicit parameter-name tokens and changes `requires_grad` at a declared release step.

The validation model freezes `auxiliary_head.weight` before step five and unfreezes it at step five. The schedule fails if its token matches no parameter, preventing a silent no-op.

Deterministic Execution

The utility

- seeds Python;
- seeds CPU PyTorch;
- seeds all visible CUDA devices;
- requests deterministic PyTorch algorithms;
- disables cuDNN benchmarking;
- enables deterministic cuDNN behavior;
- creates a dedicated seeded dataloader generator;
- seeds Python inside dataloader workers;
- hashes exact tensors in ordered batches.

Observed Fingerprints

Seed 314159 produced

- 3f2f9376bb36308f27f2a7d982f1993e42333d282cd108820fa59254a06e3c24

Repeating seed 314159 produced the same fingerprint. Seed 314160 produced:

- 7b411d9c79e0d69ced80eb4837e7da87200db8452a57d1429d8d601e20ebf626

The different seed produced a different shuffled order. PyTorch does not promise identical values across releases, platforms, or CPU/GPU device changes. The fingerprint is therefore an environment-scoped reproducibility check, consistent with the Stage 0 frozen environment contract.

Required Controls

The machine-readable control matrix freezes four controls.

ID	Control	Purpose
C0	fixed equal	raw equal-weight baseline
C1	fixed manual normalization	isolate unit scaling
C2	adaptive without physics	isolate optimizer adaptation
C3	physics with identical fixed weights	isolate added residual

The full machine identifiers are

- C0_FIXED_EQUAL ;
- C1_FIXED_MANUAL_NORMALIZATION ;
- C2_ADAPTIVE_WITHOUT_PHYSICS ;
- C3_PHYSICS_IDENTICAL_FIXED .

These controls prevent three invalid conclusions:

1. improvement from normalization cannot be credited to physics;
2. improvement from adaptive weighting cannot be credited to physics;
3. a physics residual cannot be credited without a matched fixed-weight arm.

Deterministic Smoke Problem

Model

The harness uses

- a shared two-input, one-output linear projection;
- one auxiliary linear head;
- fixed initial weights;
- six deterministic input rows.

Losses

Three components share the same trunk

- data_fit : protected regression objective;
- harmonic_shape : aligned auxiliary objective;
- weak_physics : deliberately opposing residual target.

The opposing target is intentionally artificial. Its only purpose is to prove that a negative gradient cosine is detected and projected correctly.

Initial Diagnostics

Component	Raw loss	Gradient norm
data_fit	0.6686458	1.3835733
harmonic_shape	0.4329047	0.8372000
weak_physics	1.0729164	1.8396293

Pairwise Gradient Interaction

Pair	Cosine
data versus harmonic	0.9999251
data versus weak physics	-0.9548379
harmonic versus weak physics	-0.9584037

The harmonic objective is almost perfectly aligned with data fit in this toy problem. The weak-physics objective strongly opposes both.

Gradient-Statistics Weights

At full activation

Component	Weight
data_fit	1.000000
harmonic_shape	1.6526198
weak_physics	0.7520935

This is the expected magnitude-balancing behavior: the smaller harmonic gradient receives more weight and the larger physics gradient receives less. It does not resolve the negative cosine.

ReLoBRaLo-Style Response

The initial weights were all one. In the follow-up observation:

- data loss was scaled to 50% ;
- harmonic loss was scaled to 90% ;
- physics loss was scaled to 110% .

The weights became:

Component	Follow-up weight
data_fit	0.9176718
harmonic_shape	0.9875289
weak_physics	1.0947993

The relatively slow or adverse physics progress receives more weight. This is correct adapter behavior but also illustrates why adaptive weighting alone is unsafe: it can amplify an invalid physical residual. Gradient conflict and predictive controls must remain visible.

Conflict Projection

The weak-physics cosine changed from

- -0.9548379
- to:
- 0.0000002

The residual numerical value is within floating-point tolerance of zero. The main data gradient was not modified.

Optimizer Step

After assigning the composed gradient to the shared parameters and executing one SGD step:

Update-to-parameter ratio: 0.4569128 .

The ratio is intentionally large enough for the smoke test to prove that a real update occurred. It is not a recommended training threshold.

Exit-Gate Results

Check	Result
named loss units and normalization	pass
loss values and EMA tracking	pass
per-loss gradient norms	pass
pairwise gradient cosines	pass
update-to-parameter ratio	pass
all four adapter modes	pass
staged loss activation	pass
freeze-unfreeze schedule	pass
deterministic seed and dataloader	pass
diagnostic <code>.grad</code> isolation	pass
main-preserving conflict projection	pass
all four matched controls registered	pass

Result

12 / 12 checks pass.

Durable Artifacts

Implementation

- `scripts/training/physics_guided_optimization_instrumentation.py`

Validator

- `scripts/analysis/wave_5_2r/stage2_evaluation_and_optimization_instrumentation/validate_stage2_instrumentation.py`

Machine Evidence

`Under` `output/analysis/wave_5_2r/stage2_evaluation_and_optimization_instrumentation/`

- `stage2_control_matrix.yaml` ;
- `stage2_diagnostic_records.csv` ;
- `stage2_gradient_interaction_matrix.csv` ;
- `stage2_exit_gate_summary.json` .

Human Documentation

- implementation usage note;
- validation usage note;
- Sphinx API page;
- project usage guide entry;
- this Markdown report;
- validated PDF companion.

Interpretation Boundaries

What The Pass Proves

The pass proves that

- named objectives can be normalized and tracked;
- gradient magnitude and direction can be measured independently;
- diagnostics do not mutate `.grad` ;
- all required adapters produce valid weights or projected gradients;
- schedules behave at declared boundaries;
- deterministic batch ordering can be fingerprinted;
- the required matched controls are frozen before model testing.

What The Pass Does Not Prove

The pass does not prove that

- any physics loss improves polished-setpoint forward prediction;
- gradient balancing improves generalization;
- a negative cosine always requires projection;
- a positive cosine means a residual is physically correct;
- ReLoBRaLo-style weighting should be promoted;
- conflict-aware optimization should replace fixed controls;
- the analytical anchor is stable outside its training domain.

Operational Use In Later Stages

Every later composite-loss candidate should record

1. raw and normalized loss values;
2. loss EMAs;
3. component weights;
4. per-component gradient norms;
5. pairwise gradient cosines;
6. update-to-parameter ratios;
7. schedule state;
8. seed and batch fingerprint;
9. whether projection occurred;
10. the matched control identifier.

An optimizer adapter must not be selected after looking only at held-out predictive results. The adapter comparison must be declared in the campaign plan.

Stage 3 Entry Decision

Stage 2 authorizes Stage 3

Analytical Anchor Reproduction And Stress Tests

Stage 3 should use the instrumentation to distinguish:

- analytical-anchor approximation error;
- residual-network capacity;
- loss-scale imbalance;
- gradient conflict;
- extrapolation instability.

No physics-loss campaign is authorized by Stage 2 alone. The next action is an analytical reproduction and stress-test package, not adaptive PINN training.

Conclusion

Wave 5.2R now has an optimization microscope.

The most important result is not that four adapters exist. It is that future claims can separate:

- a better physical formulation;
- a better normalized objective;
- a different gradient scale;
- a less conflicting update;
- a schedule effect;
- a true predictive improvement.

That separation is essential for the full-PINN program. Without it, an adaptive optimizer could hide an invalid equation or make a coincidental improvement look like physical knowledge. With it, Stage 3 can begin from an observable and falsifiable optimization contract.